RUBIK'S CUBE SOLVER · BUILD PLAN

7-Day Build Plan

In 7 days we build a camera-guided Rubik's Cube solver that checks every move the user makes, re-plans after mistakes, and works both on a laptop and through our XREAL One Pro glasses with the Eye camera.

DAY 1	DAY 2	DAY 3	DAY 4	DAY 5	DAY 6	DAY 7
Foundations	**Live camera**	**Real photos**	**2-view scan**	**Move tracking**	**Glasses**	**Test + demo**
Solver + colours	YOLO on webcam	Retrain YOLO	Auto capture	Check every move	Eye → glasses	Fix, rehearse
✓ DONE						

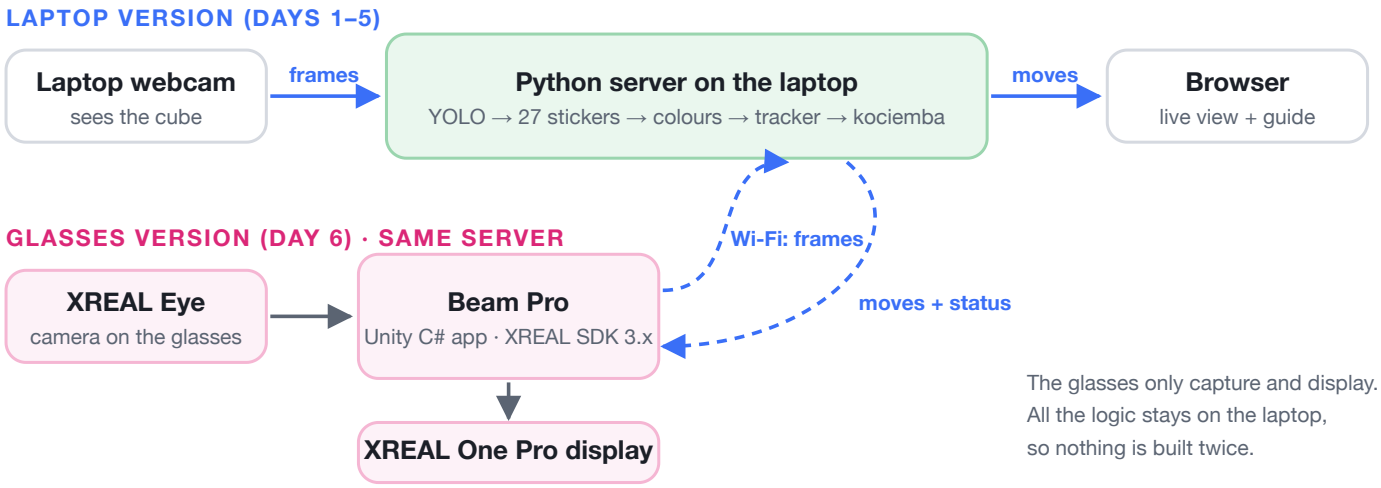
✅ Done by Day 7

- Laptop version: webcam, automatic 2-view scan, ~21-move solution, live move checking with re-planning
- YOLO retrained on real photos of our cube
- Glasses version: Eye camera → Beam Pro → laptop server → instructions shown in the glasses
- Tested in 3 lighting setups, demo rehearsed

⏭ After Day 7

- Running everything on the Beam Pro alone (no laptop): YOLO and logic ported to C#
- Turn arrows drawn on the real cube
- Licence decision before any release (kociemba GPL, YOLO AGPL)

How the system fits together



The glasses only capture and display. All the logic stays on the laptop, so nothing is built twice.

DAYS 1–4

Build the laptop version

DAY 1 Foundations · DONE

Goal: A fast, reliable solver and colour reading.

- Solver switched to **kociemba**: ~21 moves instead of ~110, in milliseconds
- Colour fallback added: centre rebalance, then **rubiks-color-resolver**
- Test harness, results and trained YOLO model moved into the repo (`bench/` , `models/`)

✅ **Done when:** Solutions ≤ 22 moves · all 43 tests pass

Who: Done

DAY 2 Live camera

Goal: See YOLO find the real cube live on the laptop.

- Webcam frames stream to the server (WebSocket `/ws/live` , ~10 per second)
- YOLO finds the 7 corner points; the 27 sticker colours are read
- Live view draws the points and colours over the video
- **"Save frame"** button to collect real photos for Day 3

✅ **Done when:** Points follow the real cube smoothly (≥ 10 frames per second)

Who: Python + browser

Glasses track starts today: Unity + XREAL SDK 3.x set up, Eye camera image showing on the Beam Pro.

DAY 3 Teach YOLO the real cube

Goal: Make YOLO reliable on our real cube, not just computer-made images.

- Record 300–500 frames: many angles, distances, lighting, hands; include some from **eye height** (how the Eye will see it)
- Labelling page: YOLO places the 7 points, a person fixes wrong ones
- Retrain YOLO on real + synthetic frames (~30 min), test on real frames it never saw

✅ **Done when:** On real photos: cube found ≥ 98%, average point error < 5% of the cube

Who: Everyone records; Python person trains

Most important day. Everything after depends on YOLO working on the real cube.

DAY 4 Scan in 2 views

Goal: Read the whole cube in two quick, automatic captures.

- View 1: Top + Front + Right. Flip the cube. View 2: Bottom + Left + Back
- Each view is captured automatically when the picture is steady
- Colours named (rebalance + resolver), cube validated, solved
- Checks: same view twice · wrong flip · back to the starting position before solving

✅ **Done when:** A real scrambled cube scans correctly ≥ 9 times out of 10

Who: Python + browser

DAYS 5–7

Tracking, glasses, demo

DAY 5 Live move tracking

Goal: Check every move the user makes and re-plan after mistakes.

- Show the move → wait until the view is steady (~5 frames) → compare with every possible move
- **Correct:** ✓ next move · **Wrong:** "You did L instead of R" + new plan · **Unsure:** wait · **Lost 3 s:** rescan
- Guide screen: current move, progress, warnings

✅ **Done when:** A full real solve with deliberate mistakes finishes and never names a wrong move

Who: Python + browser

DAY 6 Glasses connection

Goal: Use the same system through the XREAL glasses.

- Unity app on the Beam Pro sends Eye camera frames over Wi-Fi to the laptop server
- Receives the move and status back and shows them in the glasses
- Same server and logic as the laptop version, so nothing is rebuilt

✅ **Done when:** You see live move instructions in the glasses while solving

Who: Unity/C# + Python

See page 4 for exactly how the connection works.

DAY 7 Test, fix, demo

Goal: A demo that works start to finish.

- Test 5 scrambles × 3 lighting setups, on the laptop and in the glasses
- Fix bugs; save failed frames as future training data
- Rehearse the demo in the room where it will be shown

✅ **Done when:** The demo runs from scan to solved cube with no problems

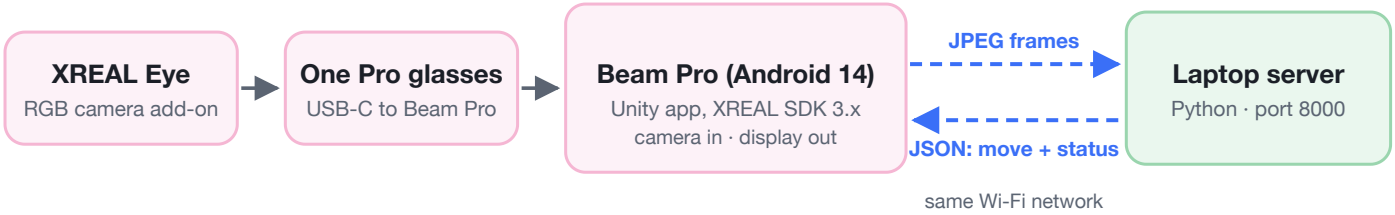
Who: Everyone

Every day ends the same way: run the tests, try it with the real cube, and show the team a short demo of what changed. If something isn't working by mid-day, say so early.

XREAL GLASSES CONNECTION

How the glasses connect

The glasses act as a camera and a screen. The Eye camera's frames travel to the laptop over Wi-Fi; the laptop sends back what to show. The laptop version and the glasses version share one server.



Build steps

	STEP	WHEN
1	**Set up Unity** Unity version supported by XREAL SDK 3.x, with Android build support. Import XREAL SDK 3.x and enable XREAL in XR settings. Build a sample scene to the Beam Pro to confirm the toolchain works.	Day 2
2	**Read the Eye camera** Use `XREALRGBCameraTexture`. Frames arrive as YUV_420_888 (three textures: Y, U, V); convert to an RGB image.	Day 2
3	**Send frames to the laptop** Resize to 640×480, JPEG-encode (quality ~80), send over a WebSocket to `ws://<laptop-IP>:8000/ws/live`. Send the next frame only after the reply arrives, so frames never pile up.	Day 3
4	**Collect Eye training photos** Use the same link with the laptop's "Save frame" feature to save real frames from the Eye's point of view for YOLO retraining.	Day 3–4
5	**Show the guide in the glasses** Read the JSON reply (current move, status, message) and show it on a panel in front of the user: move notation, a short description, ✓ / wrong-move warnings, progress.	Day 4–6
6	**Reuse the Unity scaffold** Our `xreal_ar/` project already has the C# cube model and a camera interface (`ICubeCameraSource`); its camera class targets the old NRSDK and must move to SDK 3.x.	Day 2

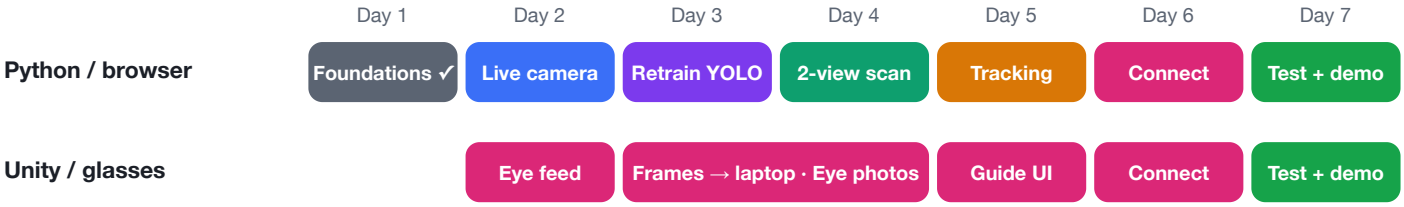
Requirements

- **Same Wi-Fi:** laptop and Beam Pro on the same network; the server listens on all interfaces (`--host 0.0.0.0`) and the laptop firewall allows port 8000.
- **Speed:** aim for ≥ 10 frames per second round trip. If Wi-Fi is slow, lower the JPEG quality or the frame size.
- **The view changes:** the Eye sees the cube from the wearer's eyes, closer and from above, which is why Day 3 includes eye-height photos and Eye frames for training.
- **Device:** XREAL warns against upgrading development devices to Android 16; keep the Beam Pro on its current version.

TEAM AND RISKS

Who does what, and what could go wrong

Two tracks in parallel



The Unity/glasses track starts on **Day 2**, not Day 6, so connecting the glasses is assembly, not a fresh start.

If a day runs late, cut these first

1. Turn arrows drawn on the cube (text instructions are enough)
2. Polish of the labelling page
3. Hard-lighting tuning (demo in normal room light)

Never cut Day 3. Real photos for YOLO decide whether everything else works.

Biggest risks

YOLO on real video	Only tested on computer-made images so far. Fix: collect frames where it fails, retrain.
XREAL setup	New SDK, device and Android builds. Fix: start Day 2, test the Eye feed first.
Wi-Fi speed	Slow network = laggy glasses. Fix: smaller frames, lower JPEG quality.
Demo lighting	Fix: test in the demo room on Day 7.

Demo-day checklist

- Laptop server running, YOLO model loaded, laptop and Beam Pro on the same Wi-Fi
- Cube scrambled; scan both views; solve with one deliberate mistake to show the re-plan
- Repeat in the glasses with the Eye camera
- Backup: the laptop version alone if the network fails

Reference documents: Implementation_Plan.pdf (architecture and rules), Benchmark_Report.pdf (why each tool was chosen), Team_Summary.pdf (short version).